High-Quality Software Engineering

Part 2: The Enemy and Our Answer (Steps 2-3)

Recap

- **Step 1**: we build **products**.
- Quality = works **and** easy to change.

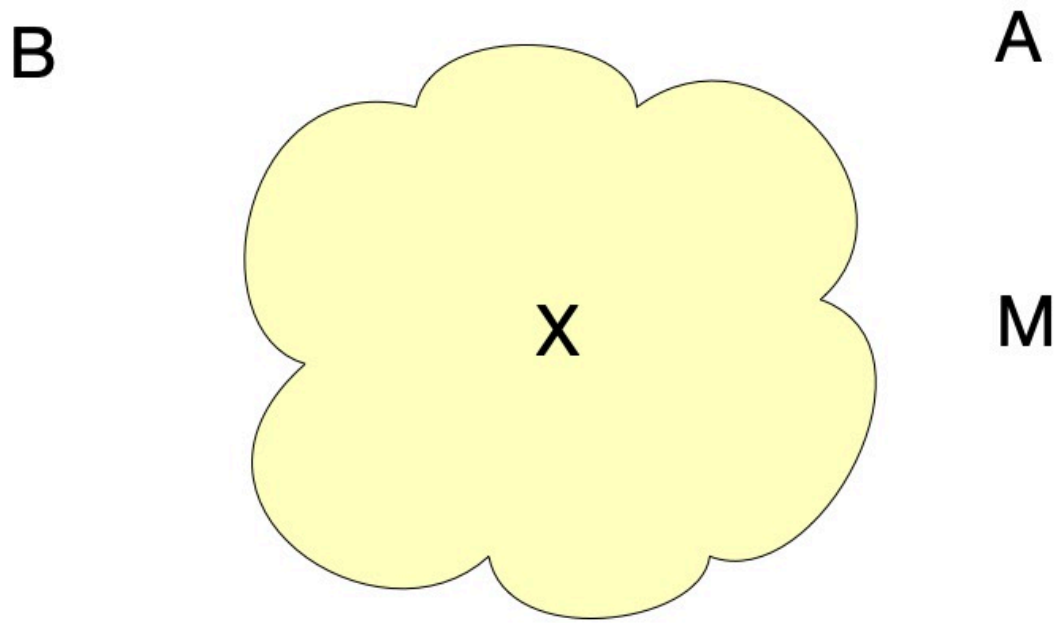
Now: **why** is that hard?

Step 2

The essence is **managing complexity**.

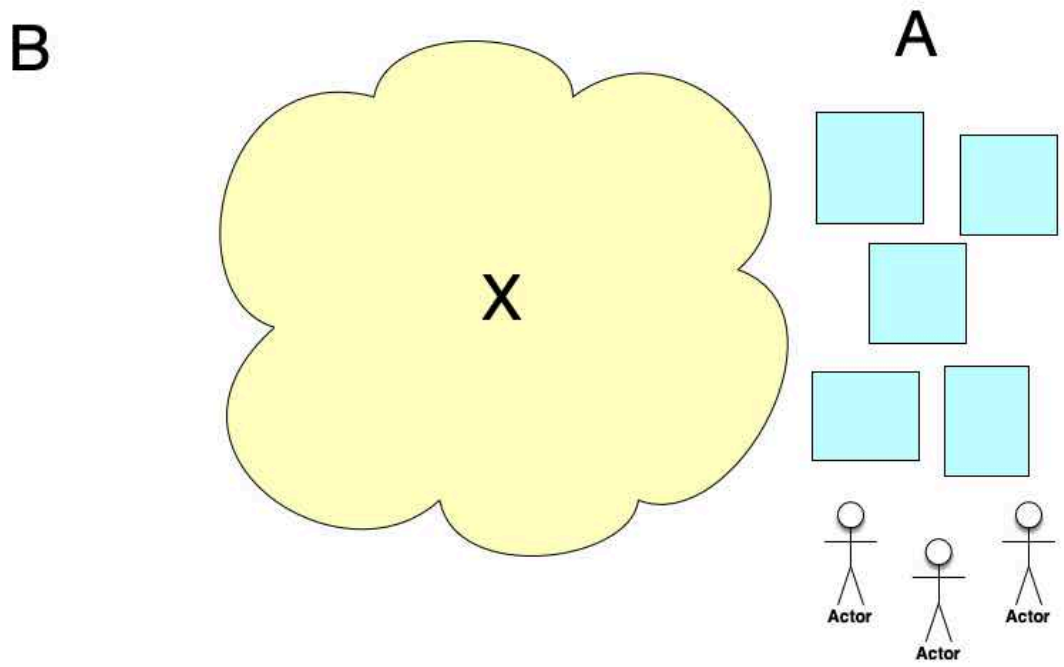
We are **problem solvers**.

The Game We Play



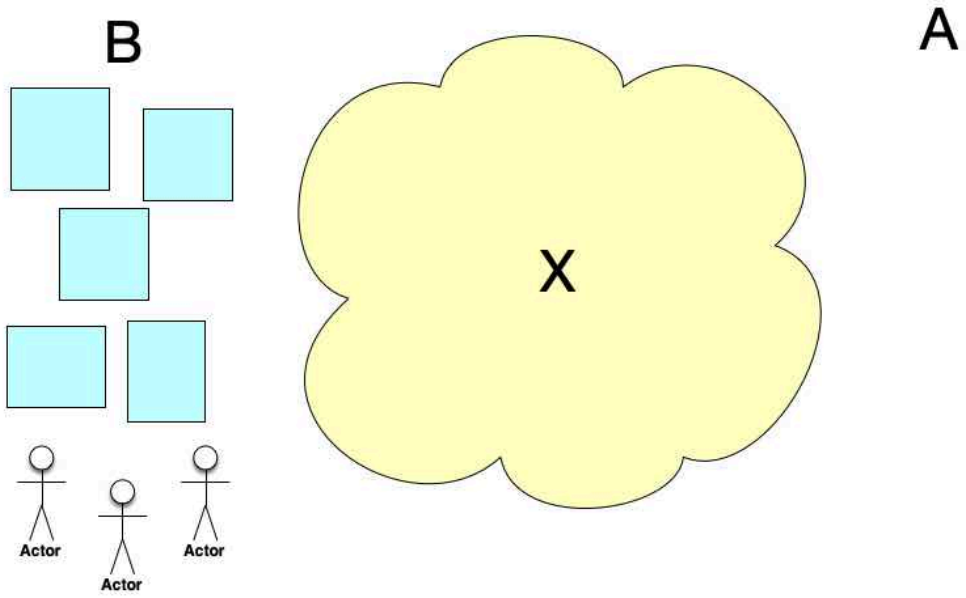
Move **M** from **A** to **B** through **X**.

Guess A, B, M, X.



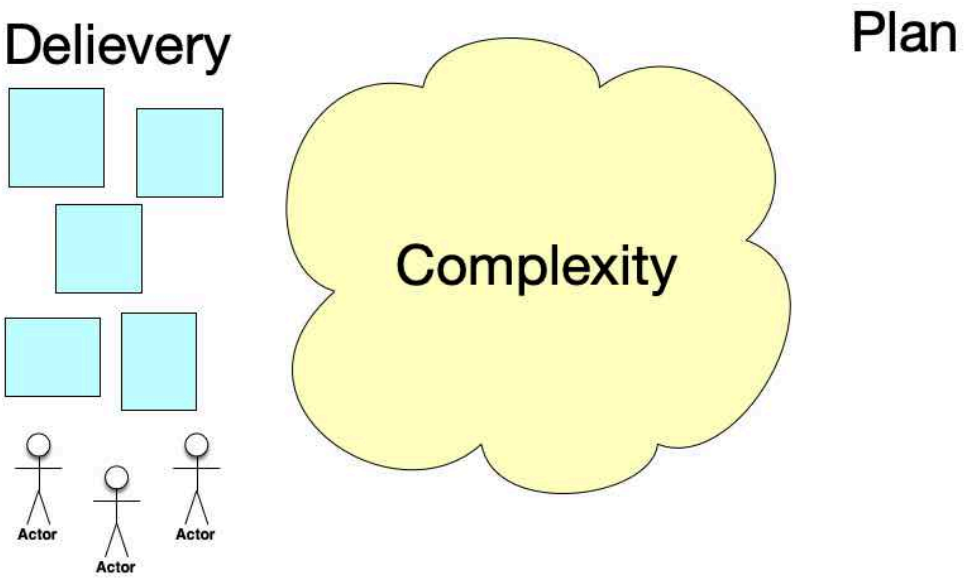
M = artifacts (code, tests, docs).

We move M in a **team**.



X looks like concrete.

It seems easy to cross.



It is a **tar pit**.

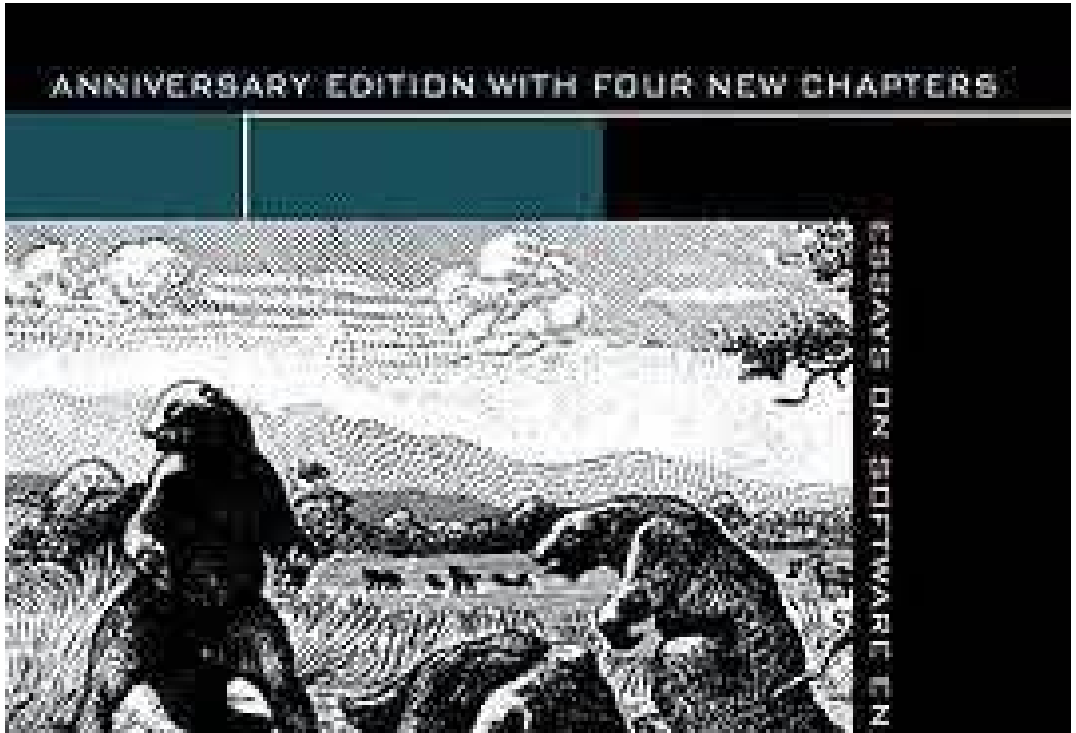
The deeper you go, the harder to escape.

The tar pit is **complexity**.

Brooks's Tar Pit

The Mythical Man-Month (1975)

- **A** = plan, **B** = delivery
- Complexity is **essential** to software.
- We cannot defeat it. We **manage** it.



A Login Story

Client: "Add **login**. Three weeks."

Sounds easy. Then:

- Where do we store users?
- How do we store passwords?
- What after 5 wrong passwords?

One Feature, Many Parts

One "simple" login touches:

- **Data** (users)
- **Security** (passwords)
- **UI** (forms)
- **Teammates** (same code)

Everything is **connected**.

Complexity Grows Fast

```
logged_in      = True or False
locked         = True or False
session_expired = True or False
# 2 * 2 * 2 = 8 states
```

6 flags = **64** states.

What Is Complexity?

Not the number of lines.

- **Dependencies**: parts need each other
- **Interactions**: one change breaks another
- **States**: many possible situations

The Name of the Game

Not coding.

Not programming.

Managing complexity.

Money, Time, People

Solve the problem under **constraints**:

1. **Money**: within budget
2. **Time**: on schedule
3. **People**: in a team

Unknown Unknowns

Two weeks with **no feedback**. Then we find:

- Students need **school login (SSO)**.
- The server has **old PHP**.

We did not know what we did not know.

Learn Early

We find unknowns by **making** something.

- **Prototype**: rough model to learn from
- **MVP**: smallest product real users can try

Cheap checks reveal expensive surprises.

Software Engineers Are...

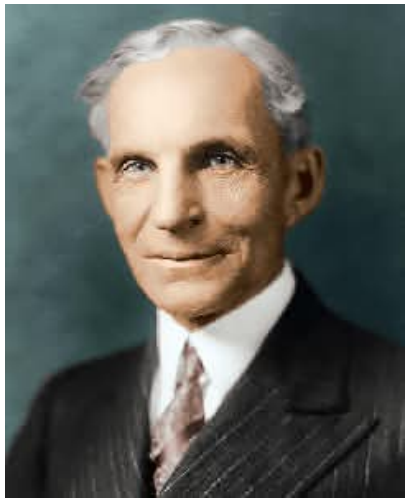
Not just coders.

Problem solvers:
technology, business, and people problems.

Two famous problem solvers: Henry Ford and Richard Feynman.

Problem Solver 1: Henry Ford

> "Whether you think you can, or you think you can't, you're right."



- Success begins with **belief**.
- Decide, **do it now**, make it **better**.

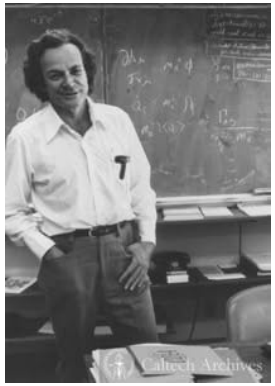
Ford's Way of Solving Problems

- Use a **system** and **process**, not whims.
- **Improve** them continuously.
- **Simplify** and **automate**.
- Decide with **rules and data**.



Problem Solver 2: Richard Feynman

> "What I cannot create, I do not understand."



- **Making** is the path to **understanding**.
- The sooner you start making, the **sooner** you succeed.

Feynman's Way of Solving Problems

1. Write down the **problem**.
2. **Think** very hard.
3. Write down the **solution**.

Can't **define** the problem? Can't solve it.

And AI?

AI writes code **fast**.

We decide if it is:

- the **right** solution
- a **safe** solution

The goal is **solving the client's problem**.

AI Is the Helper, Not the Solver

Coding is only a small part of software engineering, and AI is automating it.

- Compete with AI at coding? We **cannot win**.
- Be the **problem solver** who uses AI to implement.

Step 2 Summary

- Complexity is the **tar pit**.
- Money, time, people make it harder.
- We are **problem solvers**.

So, how do we fight it?

Step 3

Our answer: **rules** and **tools**.

Building a House

- No **tools**: slow, by hand.
- No **rules**: the house falls down.

Fighting complexity is the same.

Rules and Tools

	Rules	Tools
Meaning	How we **think**	What we **use**
Examples	KISS, DRY	Git, tests, AI

We need **both**.

Why Both?

- Tools without rules: **fast, but messy**.
- Rules without tools: **right, but slow**.

Together: **fast and right**.

Step 3 Summary

- **Rules**: how we think and decide.
- **Tools**: what we use to work.

Part 3: a closer look.
Step 4 = tools. Step 5 = rules.